

CVA6 VS-mode Invalid SFENCE.VMA Trap-Bypass Vulnerability Report

Affected version

Issue package: ../../issues/CV-5

This package was validated on a local CVA6 checkout at revision v5.3.0-168-gcd4c8b17, confirmed at commit cd4c8b17c102, using the cv64a6_imafdch_sv39 Hypervisor configuration. No fix revision has been identified yet.

Background

The issue package states the architectural decoder bug: in VS-mode, the raw encoding 0x120000f3 is accepted even though it is the rd = x1 variant of sfence.vma x0, x0. SFENCE.VMA requires rd = x0, so this encoding must raise an illegal-instruction trap.

The security PoC in this package verifies that the stale translation remains active before the invalid encoding, then shows that the invalid encoding refreshes that translation as a real SFENCE_VMA side effect.

Vulnerability type and CWE

Vulnerability type. Noncanonical privileged-instruction acceptance with a translation-management side effect, bypassing illegal-instruction trap policy.

CWE. Primary: CWE-693 Protection Mechanism Failure. Secondary: CWE-670 Always-Incorrect Control Flow Implementation and CWE-284 Improper Access Control.

Core defect

The minimal architectural issue is:

```
bad_sfence_vs:
    .word 0x120000f3
```

This encoding has the SFENCE.VMA opcode/funct fields but sets rd = x1. Entering VS-mode does not make that destination field legal. A compliant core must take an illegal-instruction trap at bad_sfence_vs.

The root cause is visible in the VS-mode decoder path. The generic SYSTEM/funct3 precheck first marks nonzero rs1 or rd as illegal:

```
if (instr.itype.rs1 != '0' || instr.itype.rd != '0') begin
    if (CVA6Cfg.RVH && v_i) begin
        virtual_illegal_instr = 1'b1;
    end else begin
        illegal_instr = 1'b1;
    end
end
end
```

Later, the SFENCE.VMA special case overwrites that earlier decision on the RVH/VS path using only the current privilege level:

```
if (CVA6Cfg.RVH && v_i) begin
    virtual_illegal_instr = (priv_lvl_i == riscv::PRIV_LVL_S) ? 1'b0 : 1'b1;
end else begin
    illegal_instr =
        (CVA6Cfg.RVS &&
```

```

        (priv_lvl_i inside {riscv::PRIV_LVL_M, riscv::PRIV_LVL_S}) &&
        instr.itype.rd == '0' ? 1'b0 : 1'b1;
end
instruction_o.op = ariane_pkg::SFENCE_VMA;

```

The non-virtual path explicitly checks `instr.itype.rd == '0'`; the VS path does not. As a result, an earlier `rd != x0` illegality can be cleared simply because the core is in virtualized S privilege.

The same decoded operation then reaches the normal fence side-effect path. At commit, `SFENCE_VMA` asserts the fence signal:

```

if (CVA6Cfg.RVS && commit_instr_i[0].op == SFENCE_VMA) begin
    if (!commit_drop_i[0]) begin
        sfence_vma_o = no_st_pending_i;
        commit_ack_o[0] = no_st_pending_i;
    end
end
end

```

In the controller, a VS-mode `SFENCE.VMA` flushes the virtual-virtual TLB path:

```

if (CVA6Cfg.RVS && sfence_vma_i) begin
    set_pc_commit_o      = 1'b1;
    flush_if_o           = 1'b1;
    flush_unissued_instr_o = 1'b1;
    flush_id_o           = 1'b1;
    flush_ex_o           = 1'b1;

    if (CVA6Cfg.RVH && v_i) flush_tlb_vvma_o = 1'b1;
    else flush_tlb_o = 1'b1;
end

```

The bug is more than fallthrough. The invalid noncanonical encoding is decoded as `SFENCE_VMA` and can execute the translation-management side effect that should have been unreachable.

Security boundary and realistic scenario

The relevant boundary is a virtualized supervisor environment where HS/M-mode monitoring, audit, binary validation, or sandbox policy relies on illegal instruction traps to reject noncanonical privileged encodings. Such software may treat canonical `SFENCE.VMA` encodings differently from malformed encodings that should never retire.

This package concerns the malformed encoding. A VS guest may have canonical ways to request translation flushes, but this raw `rd != x0` encoding should trap before it can retire. On affected CVA6 it bypasses that trap boundary and still performs the same class of translation-management side effect as a real `SFENCE.VMA`.

In a policy setting that instruments, audits, denies, or accounts for translation-management instructions by relying on architecturally required illegal traps for malformed encodings, affected CVA6 lets the guest continue past that trap point and refresh stale VS translations using the invalid encoding.

Security PoC

Assumptions

A monitor or validation layer expects malformed privileged encodings to trap and uses that trap as a policy boundary. The VS guest can update its own VS page tables, but the monitor expects the raw invalid encoding with `rd != x0` to be stopped before it can have architectural side effects.

PoC setup

The PoC enables the H extension path and enters VS-mode. It installs a VS Sv39 page table with:

- a 1 GiB identity mapping for the test code and data at 0x80000000;
- a normal readable mapping for TARGET_VA = 0x40000000;
- TARGET_VA initially pointing to old_page, which contains OLD_WORD;
- a separate secret_page, which contains SECRET_WORD;
- an initially zero attacker_buffer.

VS-mode code then performs:

```
li    t0, TARGET_VA
ld    t1, 0(t0)          /* fill translation from old_page */

/* rewrite the VS leaf PTE to point TARGET_VA at secret_page */
sd    new_leaf_pte, 0(target_l0_pt)
fence rw, rw

ld    t1, 0(t0)          /* control: must still read OLD_WORD */

bad_sfence_vs:
.word 0x120000f3         /* invalid SFENCE.VMA, rd=x1 */

ld    t1, 0(t0)          /* vulnerable CVA6 reads SECRET_WORD */
```

The pre-trigger control load is important. If the PTE rewrite had already been visible without a translation flush, the PoC exits with code 34 instead of claiming vulnerability. The vulnerable path is reachable only after the stale translation is first observed and then the invalid instruction changes the subsequent translation result.

Security result

- The guest first reads OLD_WORD through TARGET_VA.
- The guest rewrites the VS leaf PTE so TARGET_VA points at secret_page.
- Before the invalid instruction, a control load still reads OLD_WORD, so the old translation remains active.
- CVA6 accepts the invalid rd = x1 encoding and treats it as SFENCE_VMA.
- The next load from TARGET_VA reads SECRET_WORD.
- The PoC copies SECRET_WORD into attacker_buffer and exits with code 31.

CVA6 evidence

The evidence log is poc/poc.cva6-h.log:

```
poc.elf *** FAILED *** (tohost = 31) after 3130 cycles
*** [rvfi_tracer] INFO: Simulation terminated after      3118 cycles!
```

tohost = 31 is the PoC's leak_secret_after_invalid_sfence path. That path is reached only after the pre-trigger stale-translation control passes and the post-trigger load observes SECRET_WORD.

Expected result on a compliant core

The invalid SFENCE.VMA encoding raises an illegal-instruction trap at bad_sfence_vs. The M-mode trap handler verifies that the trap cause is illegal instruction and that mepc points to the invalid encoding, then exits with tohost = 0.

Observed result on CVA6

CVA6 exits with `tohost = 31`, meaning the invalid instruction did not trap and the PoC reached the path where the stale VS translation was refreshed to the new secret mapping.

Impact

- Primary: Integrity. A malformed privileged encoding bypasses a required trap/policy boundary and performs a translation-management side effect.
- Secondary: Availability and audit accuracy. Software that expects malformed encodings to trap can miss, mis-account, or fail to throttle TLB-flush activity.

Affected version status

Confirmed affected in a local run of CVA6 revision `v5.3.0-168-gcd4c8b17`, confirmed at commit `cd4c8b17c102`, using the `cv64a6_imaafdch_sv39` Hypervisor configuration.

Fixed: none identified yet.

Fix direction

The RVH/VS `SFENCE.VMA` decoder path should re-apply the instruction-specific operand legality rule instead of clearing `virtual_illegal_instr` based only on privilege:

```
if (CVA6Cfg.RVH && v_i) begin
    virtual_illegal_instr =
        (priv_lvl_i == riscv::PRIV_LVL_S && instr.itype.rd == '0') ? 1'b0 : 1'b1;
end
```

This should not mechanically preserve the earlier generic `rs1 != 0` illegality state, because legal `SFENCE.VMA` encodings may use a nonzero `rs1`. The required correction is to enforce the exact `SFENCE.VMA` operand rule for the VS path: `rd` must be zero, while `rs1` carries the address operand.